Appendix A: System Implementation Details

Candidate : Sai kumar

A.1 DeepStream Pipeline Summary

The system uses **NVIDIA DeepStream** to implement a GPU-accelerated, multi-camera inference pipeline optimized for edge deployment.

```
RTSP Streams → nvv4l2decoder → nvstreammux
→ nvinfer (Object Detection)
→ nvtracker → nvdsanalytics
→ nvinfer (Segmentation – Low FPS)
→ Sink
```

Design Rationale:
Hardware-accelerated decoding minimizes CPU load. Stream batching via `nvstreammux` enables scalable multi-camera inference. Object detection and tracking run per frame for real-time safety enforcement, while segmentation executes periodically since factory zones are static.

A.2 Model Precision & Optimization

All models are optimized using **TensorRT** for real-time edge performance.

Task	Precision	Reason
Object Detection	INT8	Maximum throughput
Segmentation	FP16	Accuracy-sensitive
Tracking & Analytics	FP16	Temporal stability
Anomaly Detection	FP32 (CPU)	Lightweight, offline

Optimizations Applied:
INT8 calibration using representative data, layer fusion, static input shapes, and TensorRT engine caching. Minor accuracy loss is accepted to ensure deterministic latency.

A.3 Edge vs Cloud Responsibilities

Function	Edge	Cloud
Inference & Alerts	Yes	No
Short-term Storage	Yes	No
Model Training (TLT)	No	Yes
Dashboards & OTA	No	Yes

Principle: All latency-critical logic runs fully on the edge; cloud is used only for non-real-time workloads.

A.4 Validation Assumptions

The design is validated using **NVIDIA DeepStream reference pipelines and published Jetson benchmarks**. Jetson hardware access was not available during development. Fixed camera placement and moderate lighting variation are assumed.